

Caching as a Separate Service

A common system design question is:

None

Why use Redis/Memcached as separate cache service?

Why not just cache inside application server memory?

Because dedicated cache services are far more scalable, reliable, and efficient.

1. What It Means

Instead of:

None

App Server stores cache in local RAM

Use:

None

App Servers



Shared Cache Service (Redis)



Database

2. Main Reason: App Services Are Stateless

Modern backend services are designed to be **stateless**.

Requests are randomly routed:

None

Request 1 → Server A

Request 2 → Server C

Request 3 → Server B

If each server has separate local cache:

None

Cache hit chance becomes low

3. Shared Cache Gives Better Hit Ratio

With separate cache:

None

All servers use same Redis cluster

So:

None

Any server can reuse previously cached data

Higher hit rate.

4. Uneven Traffic / Hot Keys

Some data becomes extremely popular:

None

Trending product

Viral post

Popular profile

Hot search term

Dedicated cache handles hot keys better.

Local cache spreads copies inefficiently.

5. Deployments Clear Local Cache

Application servers restart often:

None

New release

Autoscaling

Crash recovery

Rolling deploy

If cache is in server RAM:

None

Cache wiped frequently

Separate Redis survives deployments.

6. Independent Scaling

Need more cache memory?

None

Scale Redis cluster only

Need more API servers?

None

Scale app servers only

Independent scaling is powerful.

7. Specialized Hardware / Optimization

Cache systems can use machines optimized for:

None

Large RAM

Low latency networking

In-memory storage

Fast eviction policies

Replication

Persistence options

Better than general app hosts.

8. Request Coalescing

If 10,000 users request same uncached item:

Without shared cache:

None

10,000 app servers may hit DB

With dedicated cache:

None

Cache sends 1 request

Others wait

This is **request coalescing**.

Reduces DB overload.

9. Example Architecture

None

Clients



Load Balancer



App Servers



Redis Cache Cluster



MySQL Database

10. Why Local Cache Still Exists

Local in-memory cache is still useful for:

None

Tiny configs

Feature flags

Recently used objects

Ultra-low latency internal values

Usually combined with shared cache.

11. Multi-Level Cache Pattern

None

Browser Cache

↓

CDN

↓

App Local Cache

↓

Redis Shared Cache

↓

Database

Best performance.

12. Backend + Client Cache

Also cache on client side:

None

Browser cache

Mobile app storage

ETags

LocalStorage

Avoid network requests entirely.

13. Use CDN Too

For static content:

None

Images

Videos

JS

CSS

Downloads

Use CDN.

14. Separate Cache Advantages

None

Higher cache hit ratio

Survives deployments

Independent scaling

Better performance

Request coalescing

Centralized cache management

15. Disadvantages

None

Extra infrastructure

Network hop added

Need monitoring

Possible cache outages

Consistency challenges

16. Interview Answer

If asked “Why Redis instead of local memory cache?”

None

Because services are stateless and scaled horizontally.

Local cache fragments data across servers.

Redis provides shared cache, better hit ratio, persistence across deploys, and independent scaling.

Strong answer.

17. When Local Cache Is Enough

Use local cache only when:

None

Single server app

Small traffic

Simple prototype

Temporary values

18. One-Line Summary

Caching is usually a separate service because stateless application servers cannot efficiently share local memory caches, while dedicated cache systems

provide higher hit rates, persistence across deployments, and independent scalability.